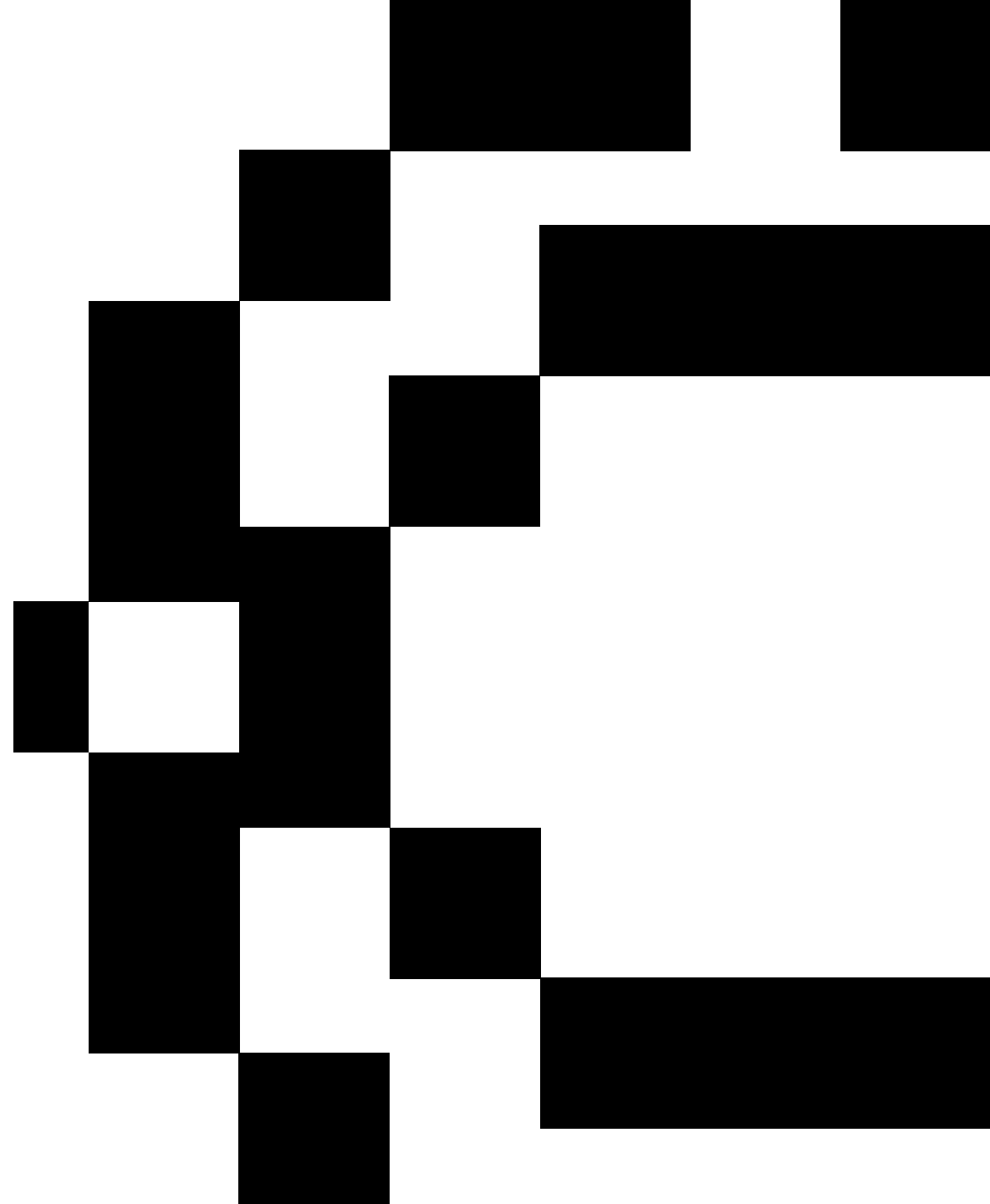


02: Model production and deployment

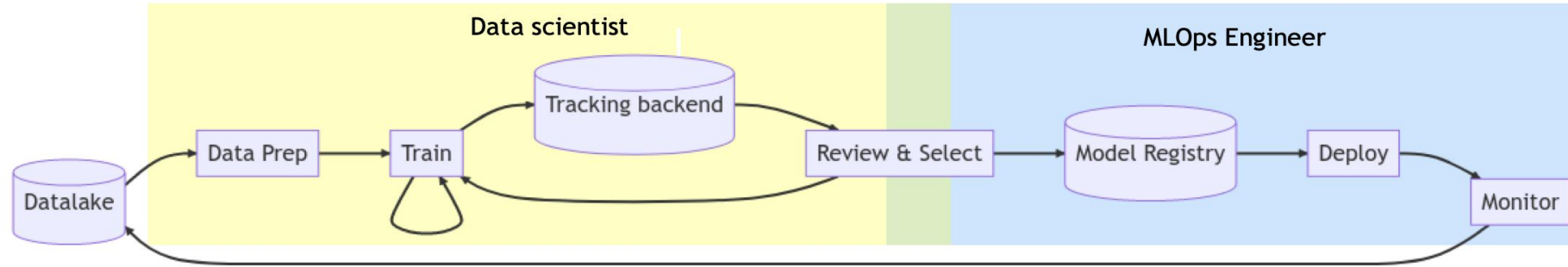
Nuno Rosa
nagostinho@novaims.unl.pt



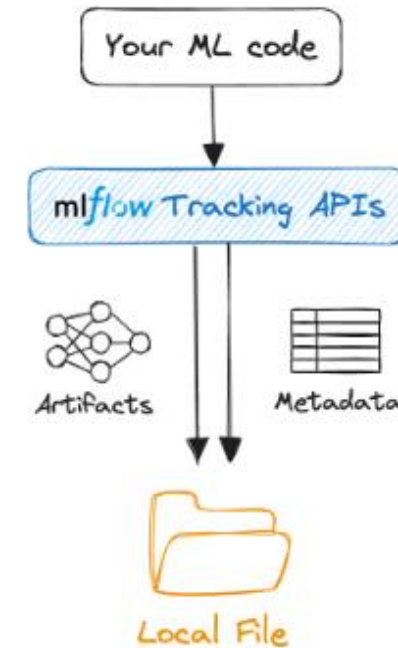
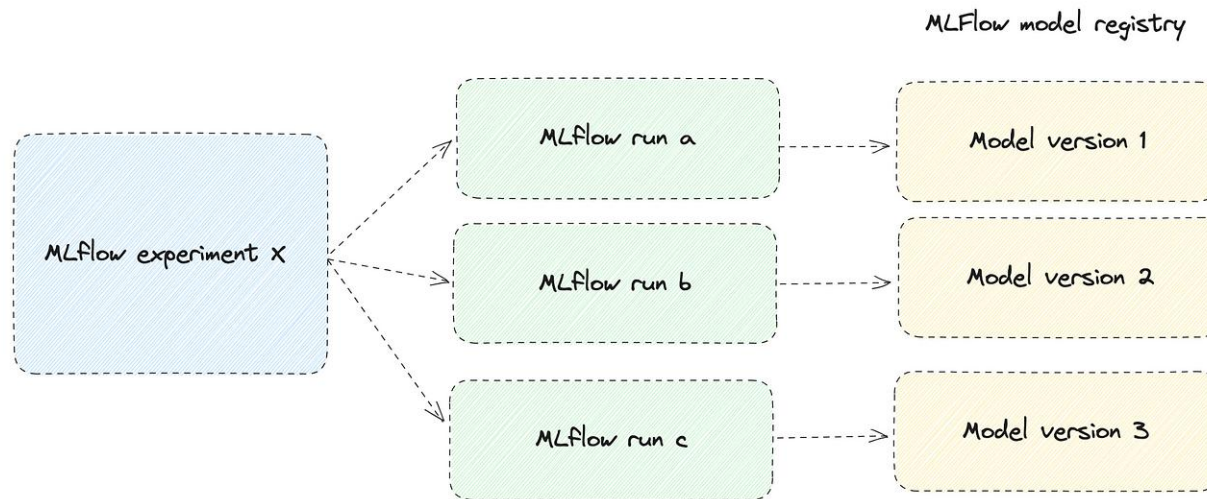
Last week recap

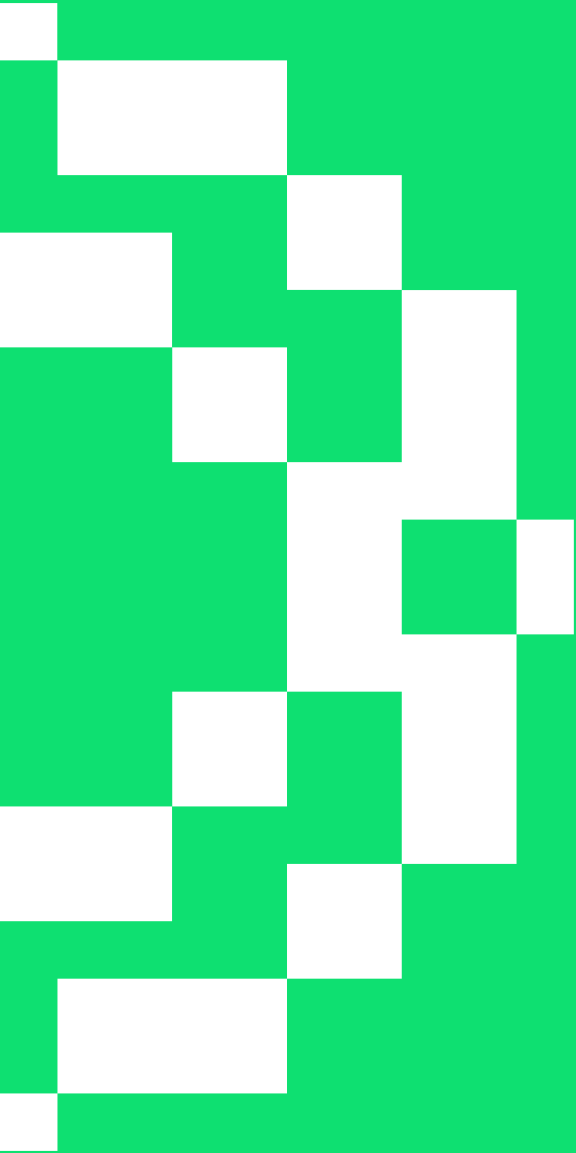
01: Model Development

MLflow Tracking



mlflow explained

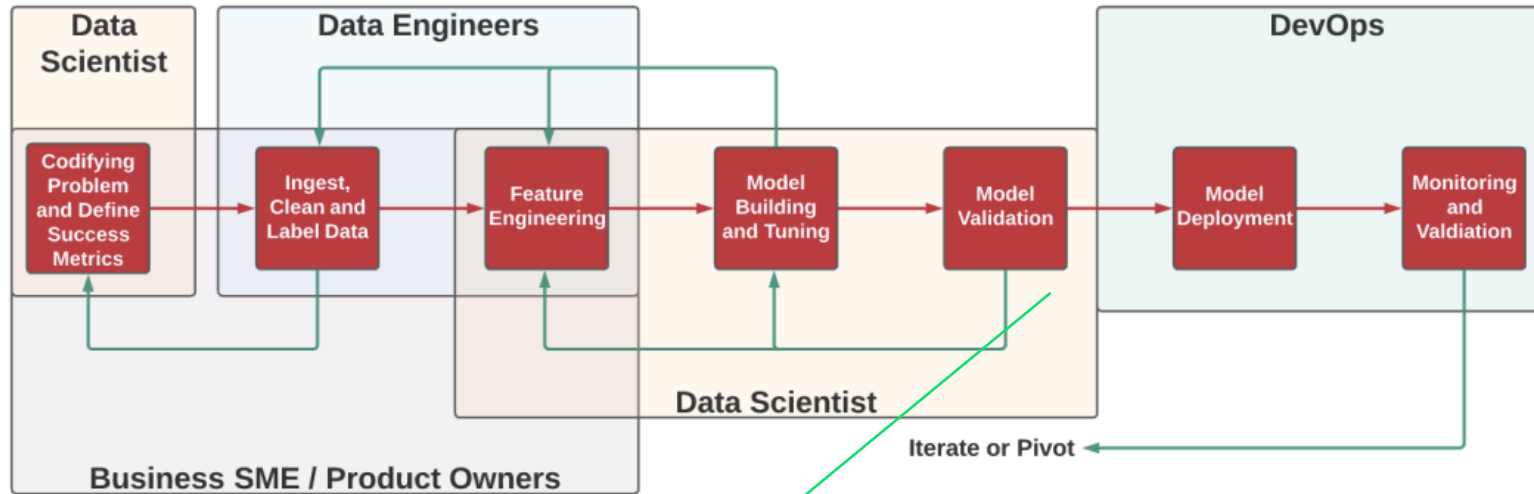




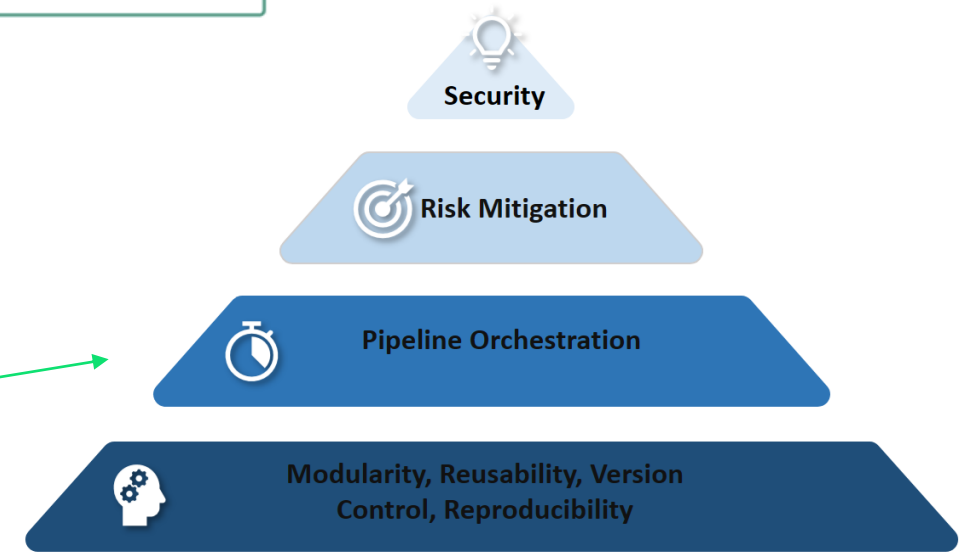
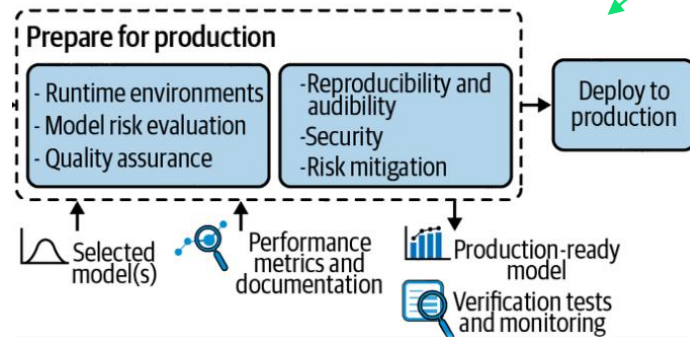
02: Model Deployment and Production

Preamble

02: Model Deployment and Production



- If it works in laboratory, we must make it work in the real world. The production environment typically is very different from the development environment -> **exponential consequences on the business and marketing part.**



02: Model Deployment and Production

```
# everything is written in one script
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

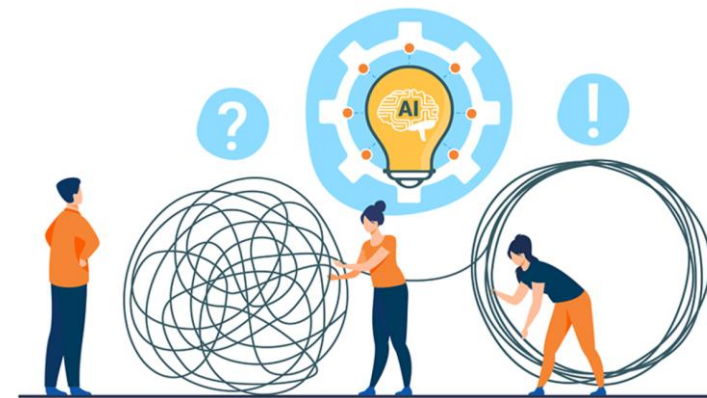
# Load and preprocess data
data = pd.read_csv('tain.csv')
data = data.dropna()
X = data[["....."]]
y = data['target']

# Train model
model = LinearRegression()
model.fit(X, y)

# Evaluate model
preds = model.predict(X)
print("metric:", mean_squared_error(y, preds))

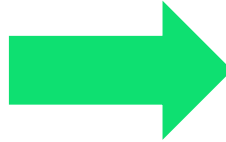
# Save model
import pickle
with open('model.pkl', 'wb') as f:
    pickle.dump(model, f)
```

Issue	Description
Hard-coded paths	Cannot generalize across environments
No version control for data/model	Can't track changes
No separation of concerns	Hard to test, debug, or scale
Manual execution	Not automatable
No logging	Cannot audit



02: Model Deployment and Production

```
# project structure:
# |— data/
# |— src/
# |   |— data_loader.py
# |   |— preprocess.py
# |   |— model.py
# |   |— train.py
# |— main.py
```



Feature	Benefit
Modular design	Easy to test and reuse
Central config/paths	Flexible deployment
Separate concerns	Scalable architecture
Parameterization possible	Supports experimentation
Can integrate with CI/CD	Automatable pipelines
Can add logging	Monitoring & traceability
Can version data/model	Auditability

```
# src/data_loader.py
import pandas as pd

def load_data(path: str):
    return pd.read_csv(path)

# src/preprocess.py
def preprocess(data):
    return data.dropna()[['...']], data.dropna()['target']

# src/model.py
from sklearn.linear_model import LinearRegression

def get_model():
    return LinearRegression()

# src/train.py
from sklearn.metrics import mean_squared_error
import pickle

def train_and_evaluate(X, y, model, save_path):
    model.fit(X, y)
    preds = model.predict(X)
    mse = mean_squared_error(y, preds)
    with open(save_path, 'wb') as f:
        pickle.dump(model, f)
    return mse

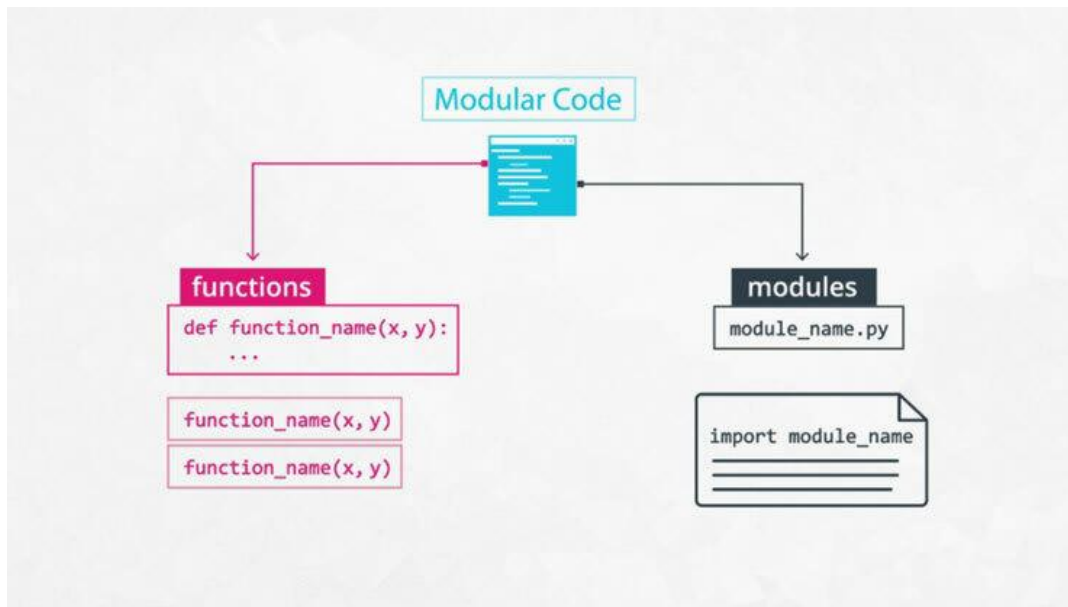
# main.py
from src.data_loader import load_data
from src.preprocess import preprocess
from src.model import get_model
from src.train import train_and_evaluate

data = load_data('data/train_20250503.csv')
X, y = preprocess(data)
model = get_model()
mse = train_and_evaluate(X, y, model, 'models/model_20250503.pkl')
print("metric:", mse)
```

02: Model Deployment and Production

Modularity, Reusability, Version Control, Reproducibility

Modular code allows you to break down complex tasks into smaller, reusable components.



Generic Template



```
02_modular_code
├── conf
│   └── settings.py
├── data
│   ├── 01_raw
│   ├── 02_intermediate
│   ├── 03_model_input
│   ├── 04_models
│   └── 05_model_output
├── docs
├── logs
├── main
│   ├── data_unit_tests_pipeline.py
│   ├── hyperparamter_tuning_pipeline.py
│   ├── ingestion_pipeline.py
│   ├── model_train_pipeline.py
│   ├── prediction_pipeline.py
│   └── preprocessing_pipeline.py
├── notebooks
├── reporting
├── src
│   ├── feature_utils.py
│   ├── model_utils.py
│   └── monitoring_utils.py
└── tests
```

DevOps

MLOps

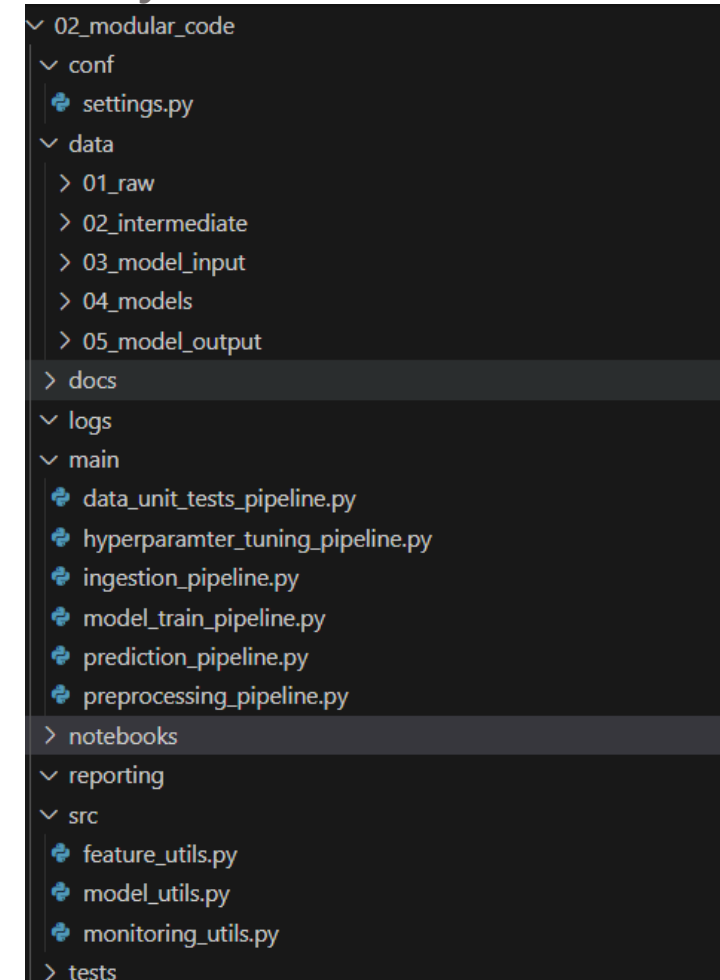
02: Model Deployment and Production

Modularity, Reusability, Version Control, Reproducibility

Modular code allows you to break down complex tasks into smaller, reusable components.

Folder/File	Description
data/	Versioned datasets, saved models and predictions
docs/	Documentation about the project/use case
logs/	Saved logs from the several processes, e.g. errors from some step
main/	Different parts of the pipeline, e.g. one file per each step of the modelling part
notebooks/	Notebooks for data/modelling/code development exploration
reporting/	Daily reports of features/ business metrics
src/	Source code files with the functions necessary to support the pipelines
tests/	Set of tests to keep code integrity

Generic
Template



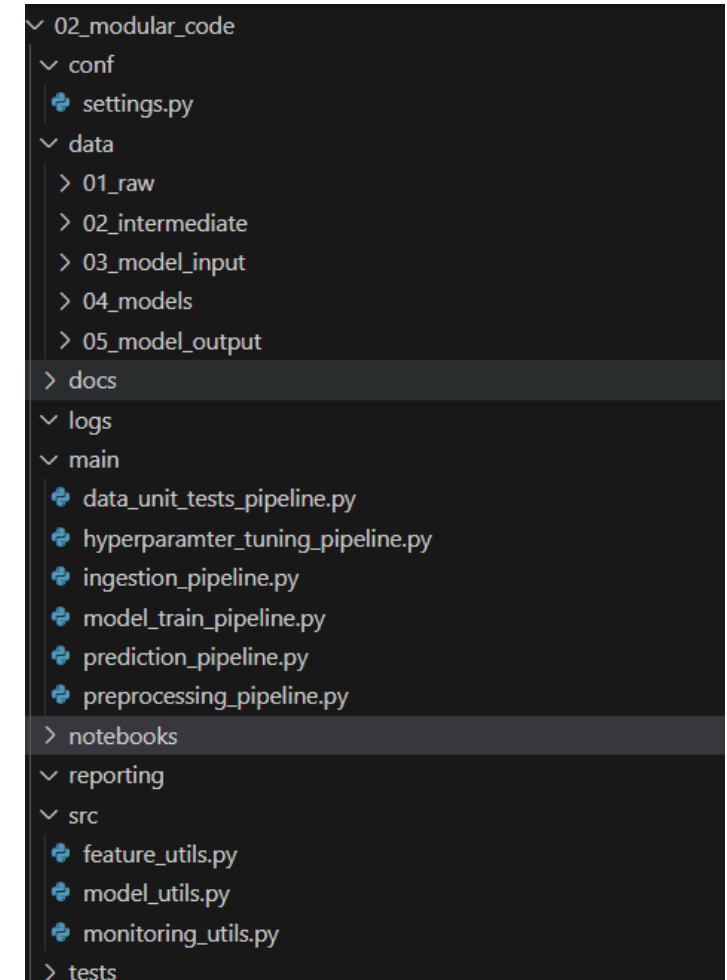
MLOps

02: Model Deployment and Production

Modular code allows you to break down complex tasks into smaller, reusable components.

- **Efficiency:** When you need to update a specific part of your pipeline (e.g., a new feature engineering technique), you can focus on that module without affecting the entire workflow.
- **Collaboration:** Multiple team members can work on different modules simultaneously, promoting collaboration and faster development.
- **Testing:** Isolating modules makes it easier to test individual components, ensuring reliability and reducing the risk of errors.

Should be included in CI/CD

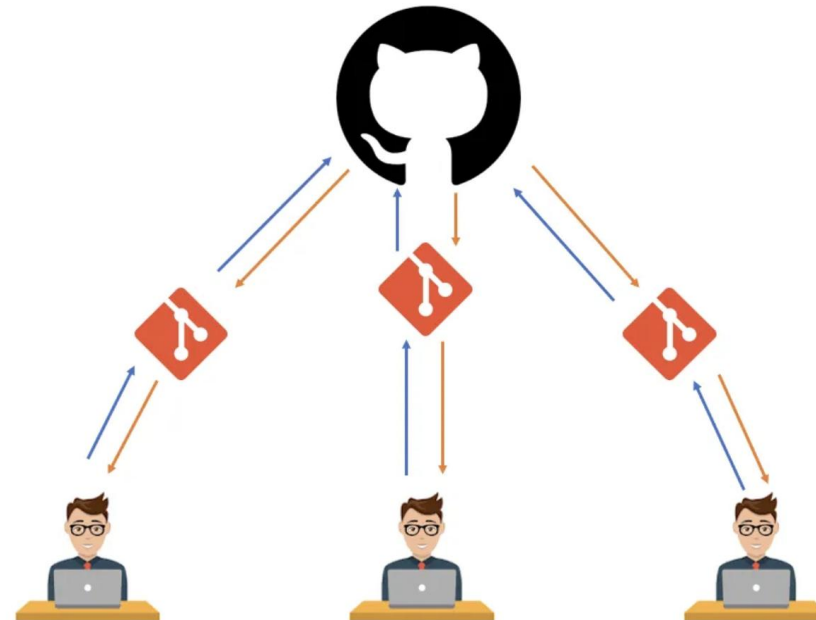


MLOps

02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility

- As a first approach, continuous integration between teams can use Git:





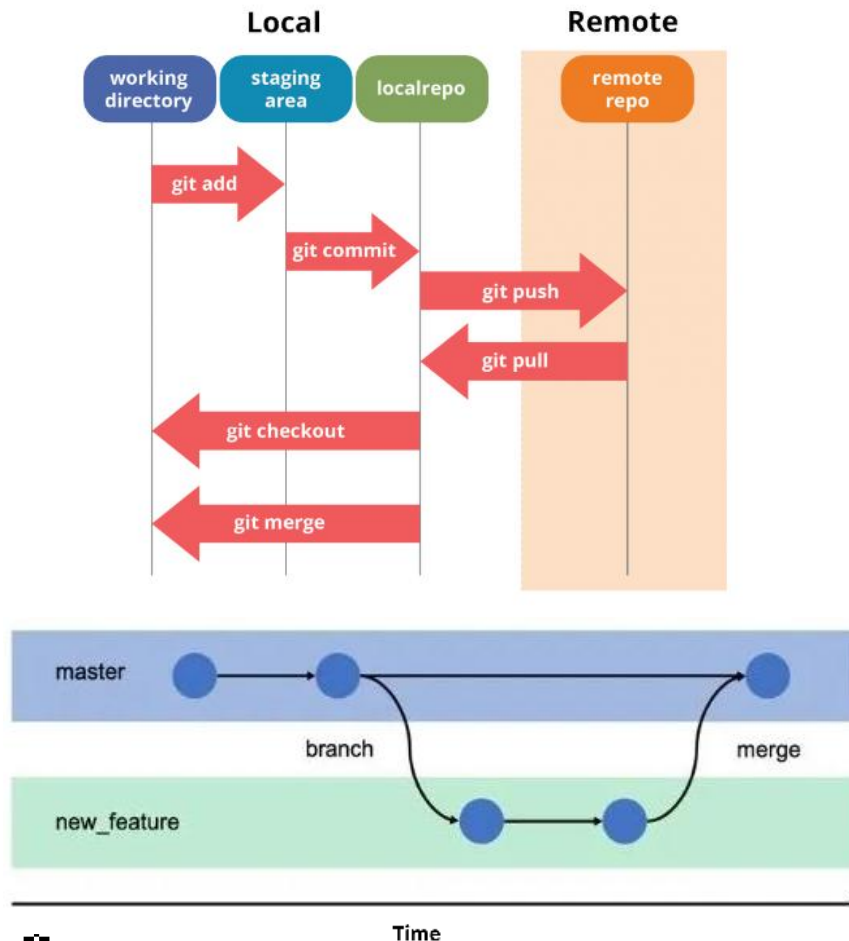
02: Model Deployment and Production

Working with Git

02: Model Deployment and Production

Continuous Integration (CI) Modularity, Reusability, Version Control, Reproducibility

Git fundamentals



1. Repositories:

- A Git repository (repo) is like a project folder that tracks changes to your files/codes over time.
- It acts as a centralized hub where collaborators can contribute and access the project.

2. Commits:

- A commit represents a snapshot of your project at a specific point in time.
- It captures changes made to files, serving as a checkpoint in your project's history.

3. Branches:

- Git allows you to create branches, which are separate lines of development.
- Each branch can have its unique changes without affecting the main project until you decide to merge them.

4. Merges:

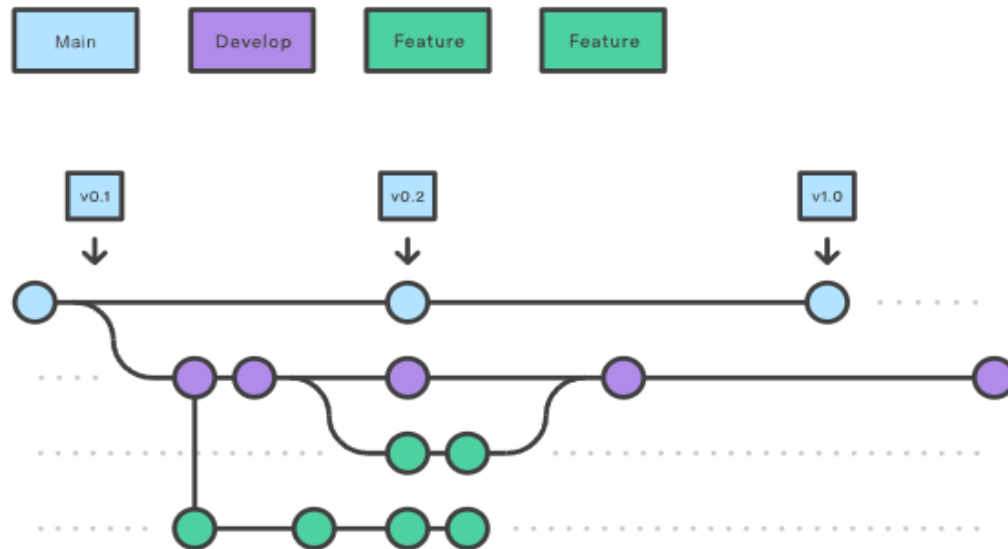
- Merging brings changes from one branch into another, combining the work done in both.
- It ensures a smooth integration of features, bug fixes, or improvements.

Git commands (there many other advanced)

1. **add:** The primary purpose of git add is to stage changes in your working directory for the next commit.
2. **commit:** Used to save the changes in your code after staging the changes with **git add**.
3. **pull:** Used to fetch and download content from a remote repository and immediately update the local repository to match that content.
4. **push:** Used to upload your local repository's changes (commits) to a remote repository.
5. **checkout:** Refers to the act of switching to a specific branch or commit.
6. **merge:** Refers to the process of combining two separate development histories into one. Git attempts to automatically merge the changes, but if there are conflicts, user intervention is needed.

02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility



- We start by having a **Main branch** - where is our production code.
- We want to work on some evolution, so we create a **Develop branch**.
- When a part of the pipeline needs to be changed, we create a **Feature branch**.

Features **should never interact** directly with Main



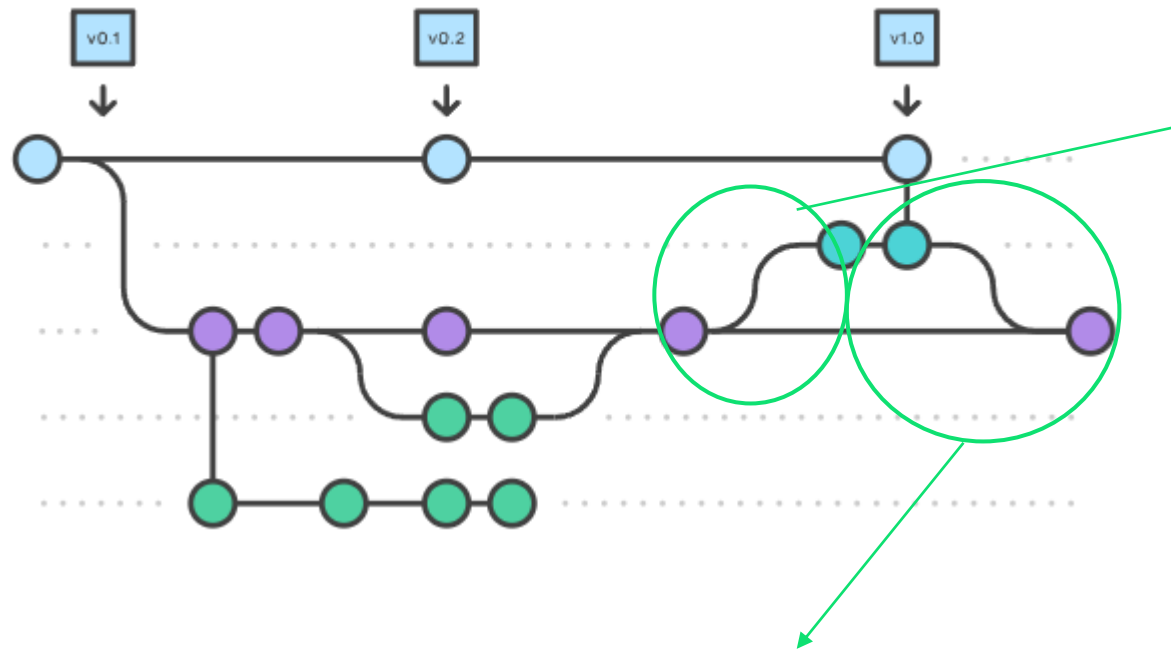
When a feature is complete, it gets **merged back into develop**.

```
✓ main
├── data_unit_tests_pipeline.py
├── hyperparamter_tuning_pipeline.py
├── ingestion_pipeline.py
├── model_train_pipeline.py
├── prediction_pipeline.py
└── preprocessing_pipeline.py
```

The main idea is that the **team can work in independent Features** to change components of the pipeline

02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility

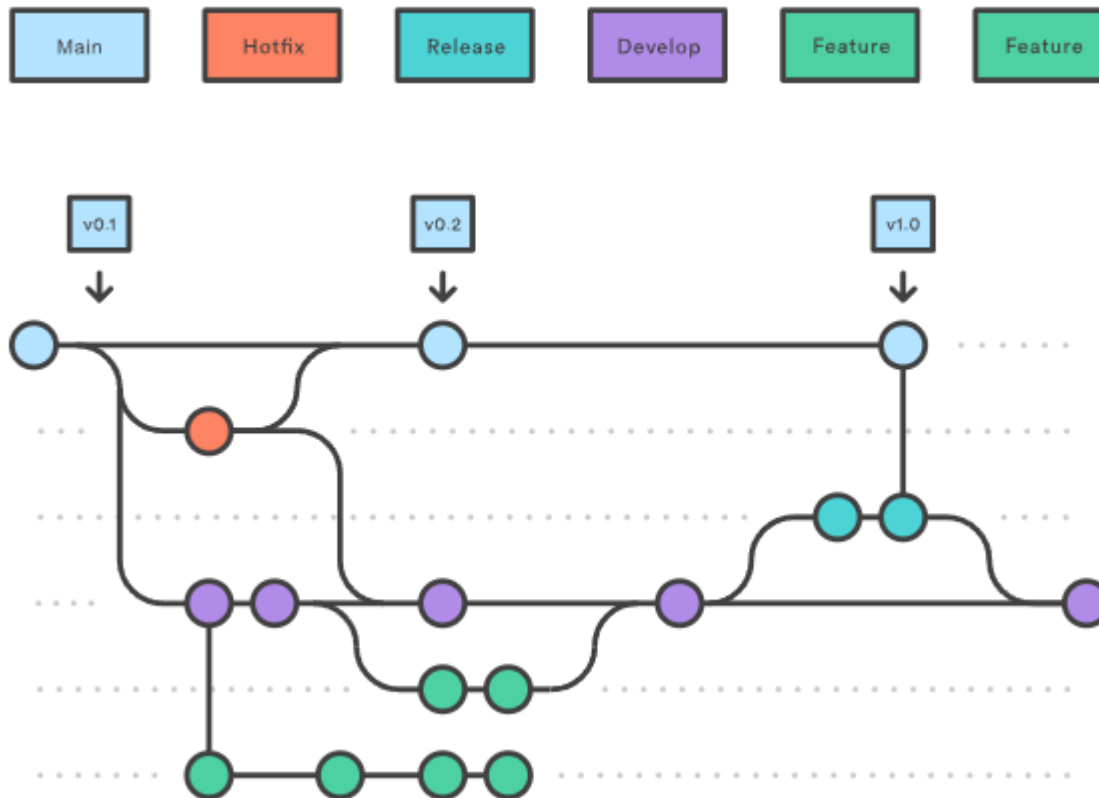


- Once develop has acquired enough features for a release, you fork a Release branch from Develop branch.
- Another team continues working on features for the next release.

- Creating this branch starts the next release cycle. Once it's ready to ship, the Release branch gets merged into Main and tagged with a version number.
- After that, it should be merged back into develop, which may have progressed since the release was initiated.

02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility



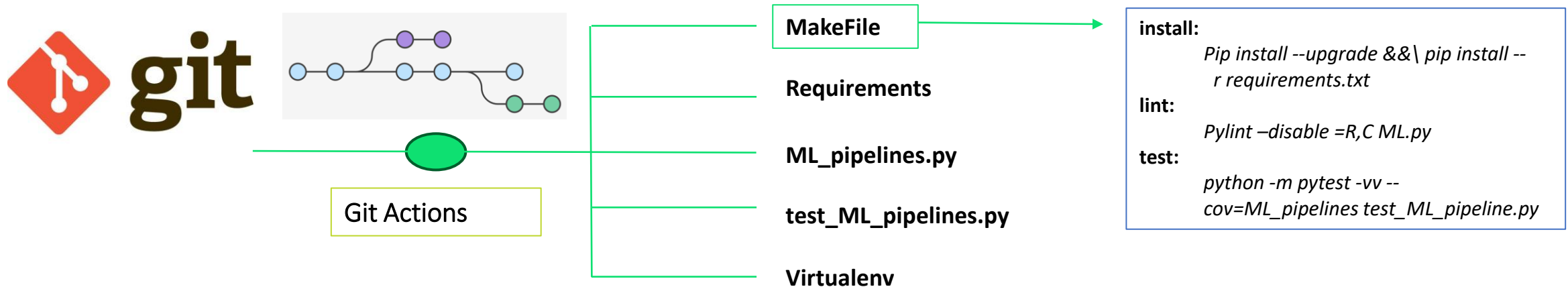
- **Hotfix branches** are used to quickly patch production releases.
- Hotfix branches are similar to **Release branches** and **Feature branches** except they're based on Main instead of Develop branch.



Bug fixes in the code (such as typo in feature type, or wrong assignment of a function).

02: Model Deployment and Production

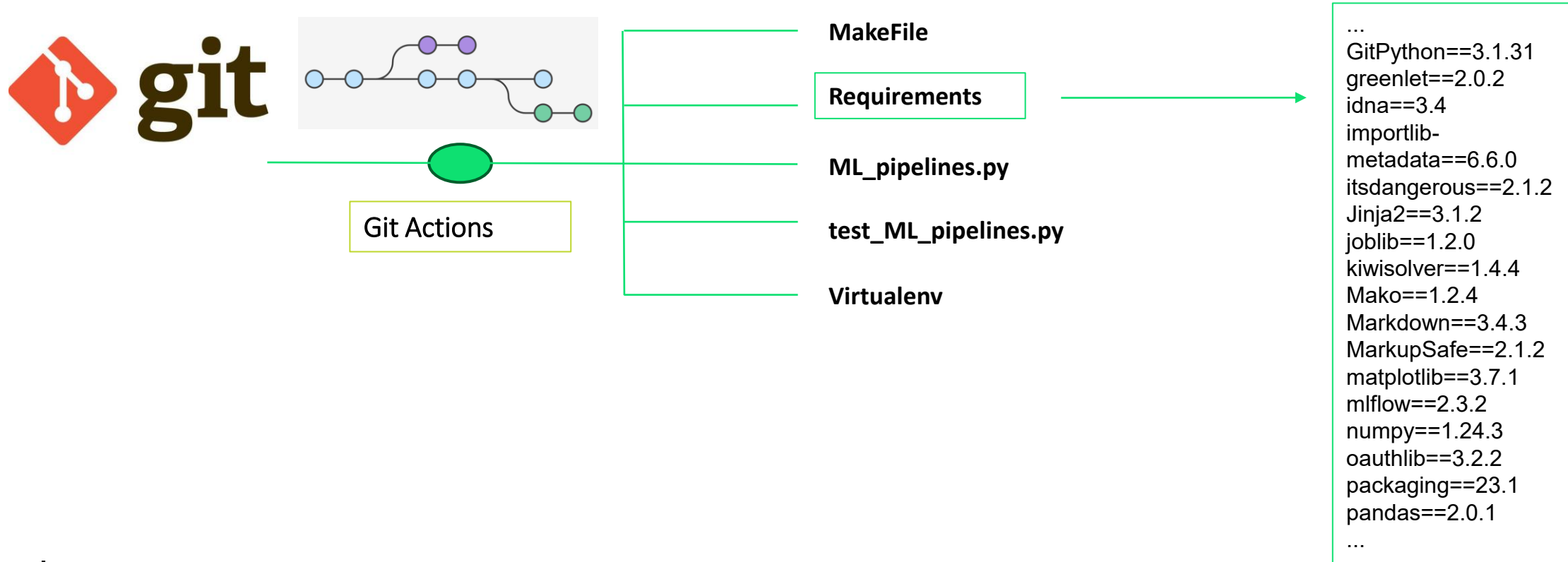
Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility



02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility

- Extra information to add **Reproducibility** to your modular code already versioned with Git:

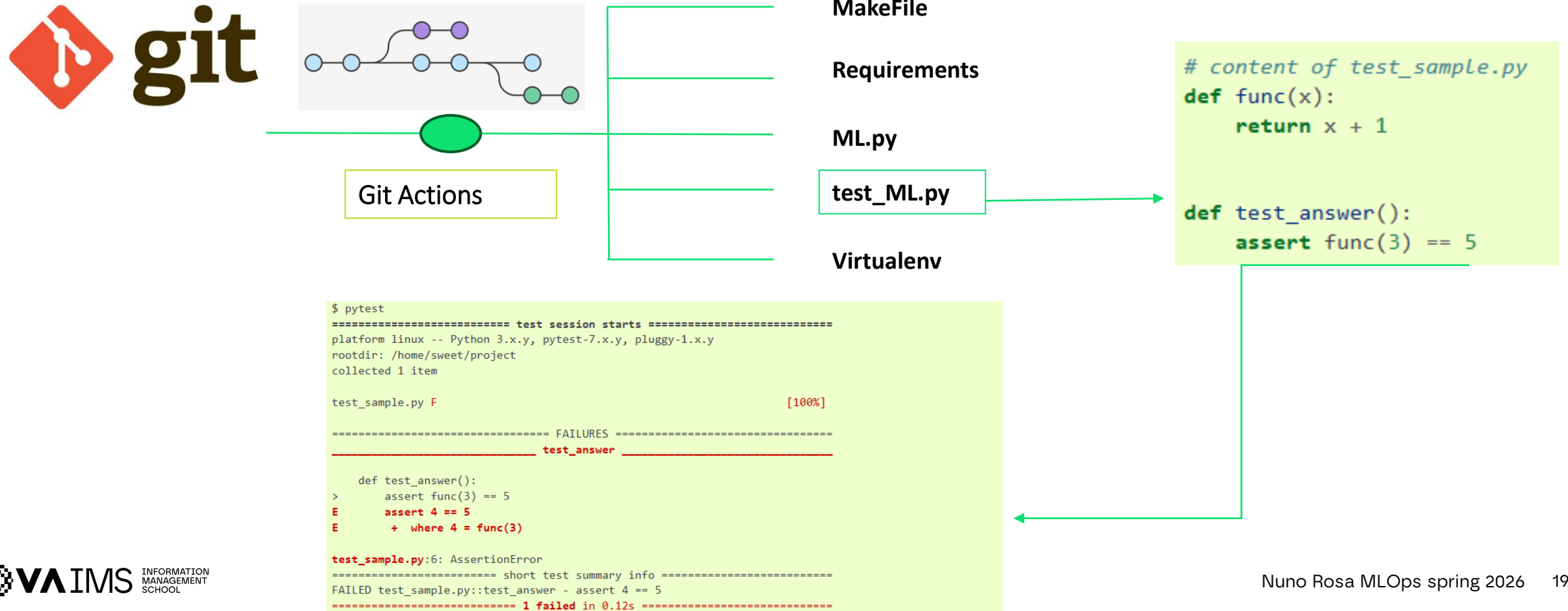


02: Model Deployment and Production

Continuous Integration (CI)

Modularity, Reusability, Version Control, Reproducibility

- Extra information to add **Reproducibility** to your modular code already versioned with Git:



02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility

```
# src/custom_preprocessing.py
def clean_data(df):
    df = df.copy()

    for col in df.columns:
        if df[col].dtype == object:
            df[col] = df[col].apply(lambda x: x.lower() if isinstance(x, str) else x)
        elif df[col].dtype in ['int64', 'float64']:
            mean_val = df[col].mean()
            df[col] = df[col].fillna(mean_val)

    return df
```



Before push to the main code, run the pytest

```
$ pytest test_custom_preprocessing.py
# ✓ All tests pass
```

- Avoid silent errors or unexpected transformations
- Guarantees the same logic is applied every run
- Fail fast if logic breaks due to schema change
- Run tests before data is used in training -> reproducible results.

```
# test_custom_preprocessing.py
import pandas as pd
import numpy as np
from src.custom_preprocessing import clean_data

def test_clean_data():
    df = pd.DataFrame({
        "name": ["Alice", None, "CHARLIE"],
        "age": [25, None, 35],
        "score": [90, 85, None]
    })

    cleaned = clean_data(df)

    # Check that names are lowercase and missing name is unchanged (None)
    assert cleaned["name"].iloc[0] == "alice"
    assert cleaned["name"].iloc[2] == "charlie"

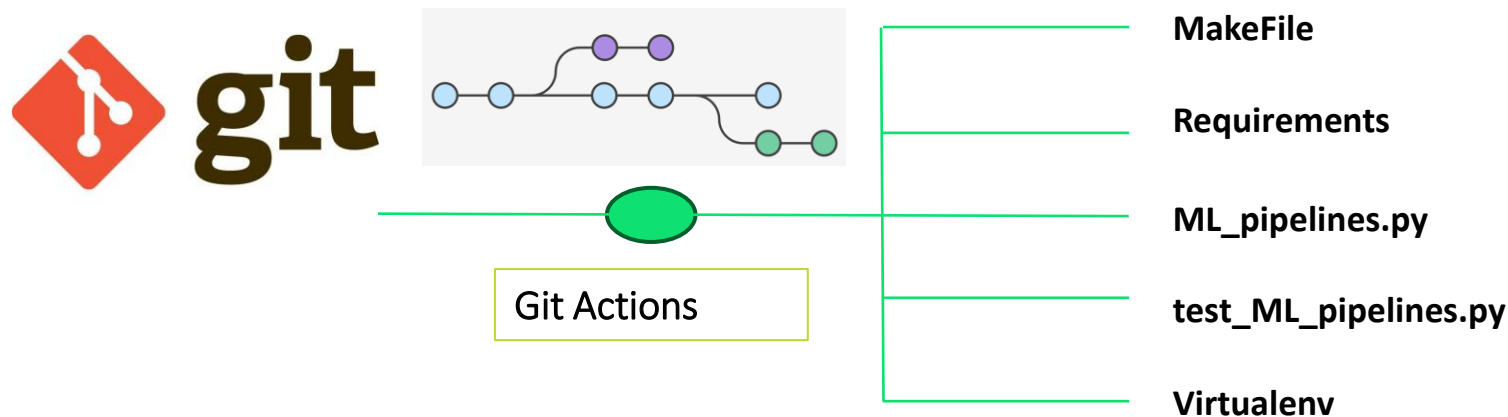
    # Check that missing age and score are filled with mean
    expected_age = (25 + 35) / 2
    expected_score = (90 + 85) / 2
    assert cleaned["age"].iloc[1] == expected_age
    assert cleaned["score"].iloc[2] == expected_score

    # Check types remain consistent
    assert cleaned["age"].dtype == float
```

02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility

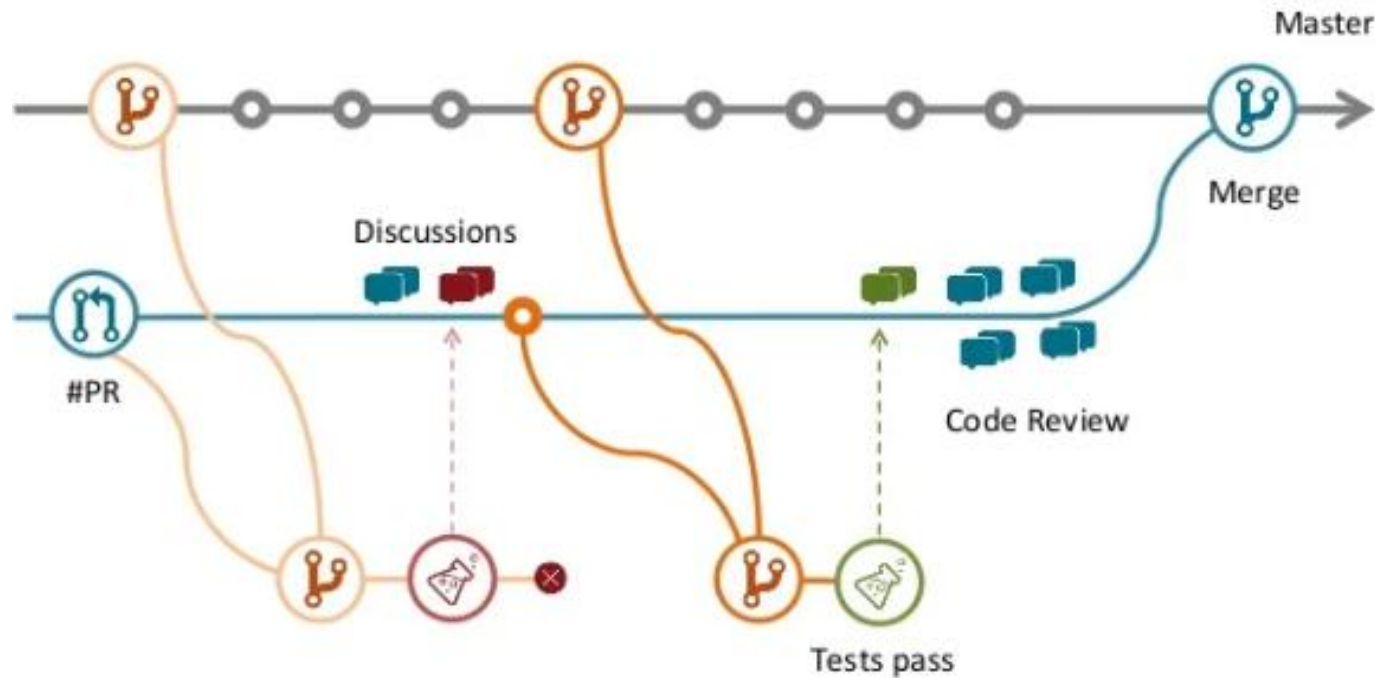
- Extra information to add **Reproducibility** to your modular code already versioned with Git:



1. A **Makefile** runs “recipes” via the make system, which comes with Unix-based operating systems. Therefore, a Makefile is an ideal choice to simplify the steps involved in continuous integration.
2. A **requirements.txt** file is a convention used by the pip installation tool, the default installation tool for Python.
3. Our **pipeline** to produce the results of the ML model.
4. Run **tests for integrity** of our pipeline during **Continuous Integration** and before each push/merge to the Main branch.

02: Model Deployment and Production

Continuous Integration (CI)
Modularity, Reusability, Version Control, Reproducibility



- In this should be the good practices in our ML framework with the respect of the development of our pipeline for modelling and deployment:
 - ✓ **New branch from master** (development component).
 - ✓ **Test** for every new function.
 - ✓ **Code reviewers** for each pull request.

02: Model Deployment and Production

```
name: Continuous Integration (CI) Pipeline

# 1. THE TRIGGER: When does this automation run?
on:
  push:
    branches:
      - main
      - develop

jobs:
  run-tests:
    runs-on: ubuntu-latest

    steps:
      - name: Check out repository code
        uses: actions/checkout@v3

      - name: Install Python and dependencies
        run: |
          pip install -r requirements.txt

      # 2. THE ACTION: Run the automated tests!
      - name: Execute Pytest
        run: pytest tests/
```

- **No more manual testing:** This file (.github/workflows/ci.yml) acts as the "glue" between your Git repository and your code quality.
- **Automatism of processes:** Every time a developer uses git push, a server in the cloud automatically wakes up, installs the environment, and runs pytest.
- **Production safety:** If the tests fail, the pipeline flashes red, and the broken code is prevented from merging into the main project.

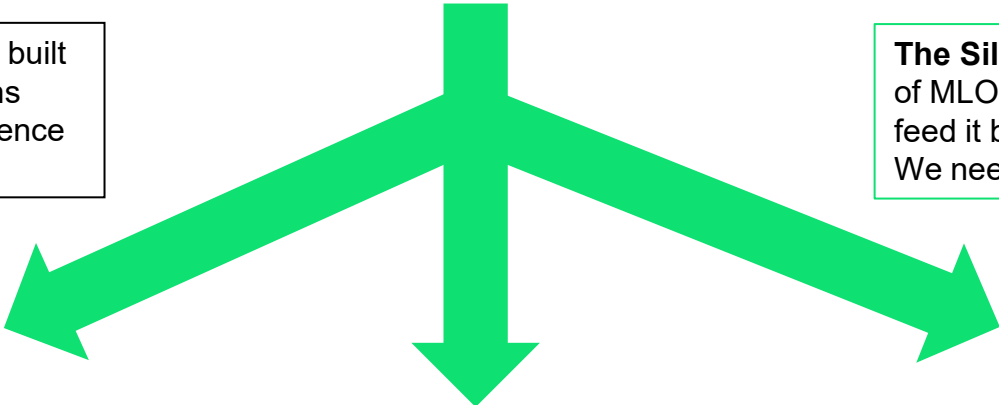
02: Model Deployment and Production

Traditional Software development

Code logic and system architecture
(**deterministic** vision of outputs)

The Paradigm Shift: Traditional software is built on rules. Machine learning is built on patterns extracted from data. This fundamental difference requires a new approach to testing.

The Silent Failure: This is the most dangerous aspect of MLOps. A model API won't necessarily crash if you feed it bad data; it will happily return a bad prediction. We need data to catch this before it happens..



	Code Testing (Unit/Integration)	Data Testing (Validation)	Model Testing (Evaluation)
Type of check	Does the Python logic work correctly?	Is the incoming data structured exactly as expected before we train or predict?	Is the mathematical output of the model acceptable?
Example	Does our <i>clean_data()</i> function successfully convert all strings to lowercase without crashing on Null values? -> pytest, unittest	Did the upstream database suddenly drop the customer_age column? Are there unexpected negative numbers in the price column? -> Great Expectations	Did the new Feature branch drop our F1-score below 85%? Is the inference latency taking longer than 30 milliseconds?

CI/CT/CD: Notice the addition of **Continuous Training** (CT) in MLOps. Because the real-world changes, models degrade over time. MLOps pipelines must automatically retrain and test models on new data to maintain performance.

02: Model Deployment and Production

Git & GitHub Actions in MLOps

Version everything. Automate everything. Ship ML with confidence.

1. DEVELOP & VERSION

Code, configs, data pipelines, models – everything in Git.



GIT REPOSITORY

```
src/
configs/
data/
notebooks/
tests/
models/
.github/
  workflows/
    mlops.yml
requirements.txt
README.md
```



1. TRACK EVERYTHING

- Code
- Data & Schemas
- Models
- Experiments
- Configs



2. AUTOMATE WITH GITHUB ACTIONS

CI/CD pipeline for ML



TRIGGER

On push / Pull Request / Schedule



SETUP

Set up Python, install dependencies, cache



QUALITY CHECKS

Linting, formatting, type checks



TESTS

Unit tests, integration tests



DATA VALIDATION

Great Expectations / data checks



TRAIN

Train model, log params & metrics



EVALUATE

Evaluate model performance



PACKAGE MODEL

Save model artifact, model card



PUSH ARTIFACTS

Push to registry, update metadata



NOTIFY

Comment on PR / Send Slack notification



mlops.yml

3. ARTIFACTS & REGISTRY



4. DEPLOY

Deploy the right model version



BENEFITS

- ✓ Version control for reproducibility
- ✓ Automated CI/CD for ML
- ✓ Consistent, reliable, and auditable ML delivery
- ✓ Faster iteration, safer deployments

1. Develop & Version (The Foundation)

- All project assets—code, configurations, data pipelines, and models—are version-controlled in a central Git repository.
- Maintains a structured directory (e.g., src/, data/, models/, .github/workflows/) to track experiments, schemas, and configurations.

2. Automate with GitHub Actions (The CI/CD Pipeline)

- Automated workflows are defined by a central configuration file (e.g., mlops.yml).
- Pipelines are triggered automatically by events like a code push, a Pull Request, or on a set schedule.
- Executes a sequential, automated process: Environment Setup → Quality Checks & Linting → Unit/Integration Testing → Data Validation → Model Training → Evaluation → Packaging → Pushing Artifacts → Notifications.

3. Artifacts & Registry (Asset Management)

- Outputs are pushed to secure cloud storage (e.g., S3, GCS).
- Utilizes tracking servers (like MLflow) to log experiments and a versioned Model Registry to manage approved models.

4. Deploy (Serving the Model)

- The approved model version is deployed to production using flexible architectural patterns: Batch processing, REST APIs, or Serverless functions.

02: Model Deployment and Production

mlops.yml — Example

Declarative ML pipeline configuration

```
mlops.yml
1 version: "1.0"
2 project: churn-prediction
3 description: "End-to-end MLOps pipeline for churn model"
4
5 paths:
6   data_dir: data/
7   models_dir: models/
8   outputs_dir: outputs/
9   reports_dir: reports/
10
11 data:
12   source: s3://mlops-bucket/raw/churn/
13   type: parquet
14   expected_schema: schemas/churn_schema.json
15   validation:
16     rules_file: configs/data_validation.yml
17
18 features:
19   feature_store: feast
20   repo: feature_repo
21   entities:
22     - customer_id
23   feature_views:
24     - name: customer_agg
25       ttl: 1d
26
27 training:
28   algorithm: xgboost
29   experiment_tracking: mlflow
30   parameters:
31     n_estimators: 300
32     learning_rate: 0.05
33     max_depth: 6
34   cv: 5
35   metrics:
36     - name: roc_auc
37       goal: maximize
38     - name: f1
39       goal: maximize
40   output_model_path: models/model.pkl
41
42 evaluation:
43   evaluate_on: test
44   threshold: 0.5
45   save_report: true
46
47 deployment:
48   target: sagemaker
49   endpoint_name: churn-prediction-endpoint
50   instance_type: ml.m5.large
51   initial_instance_count: 1
52   model_artifact: models/model.pkl
53   container_uri: 683313688378.dkr.ecr.us-east-1.amazonaws.com/churn-prediction:latest
54   environment:
55     - name: LOG_LEVEL
56       value: INFO
57
```

- PROJECT INFO**
Global settings for the MLOps pipeline.
- PATHS**
Common directories used across the pipeline.
- DATA**
Data source, format, schema and validation rules.
- FEATURES**
Feature store configuration and feature views.
- TRAINING**
 - Algorithm & parameters
 - Experiment tracking
 - Cross-validation
 - Metrics to optimize
 - Output model artifact
- EVALUATION**
How to evaluate the model and generate reports.
- DEPLOYMENT**
Where and how to deploy the model for inference.

This file drives the GitHub Actions MLOps pipeline:

- Trigger on push / PR / schedule
- Validate data & code
- Train, evaluate & log metrics
- Package model & push artifacts
- Deploy & monitor

This configuration file directly drives CI/CD tools (like GitHub Actions) to automatically:

- *Trigger* on code pushes or pull requests.
- *Validate* data and code integrity.
- *Train, Evaluate, and Log* model metrics.
- *Package and Deploy* the final artifact to production.



02: Model Deployment and Production

Modularity and orchestration

02: Model Deployment and Production

From modules to pipelines: the need of orchestration



- **We have the pieces, but who is the conductor?**



- We built modular code (src/ folder).



- We enforced quality with CI and pytest.



- **The Missing Link:**
How do we connect `clean_data()` to `train_model()` without writing messy, hard-coded "glue" scripts?



- **The Data Passing Problem:**
If Step A outputs a dataframe, how does Step B load it without hardcoding `pd.read_csv("temp_data.csv")` deep inside the logic?

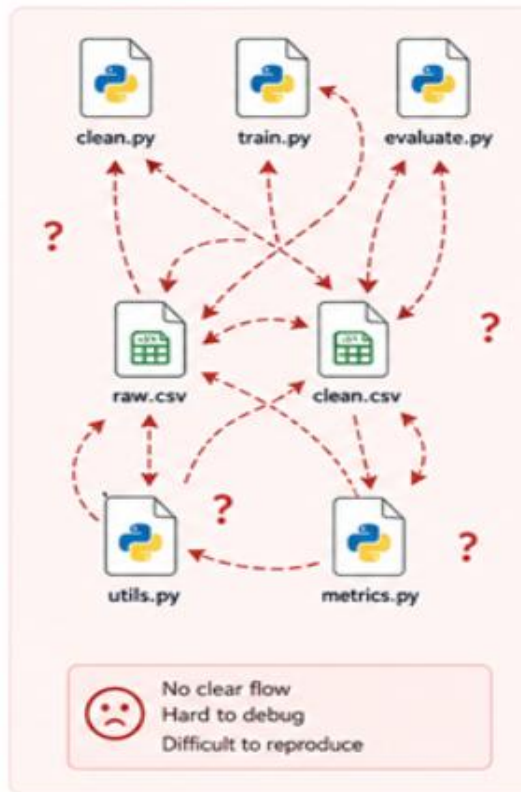


- **The Execution Problem:**
If a pipeline has 20 steps and fails at Step 15, how do we resume from Step 15 without rerunning the whole thing?

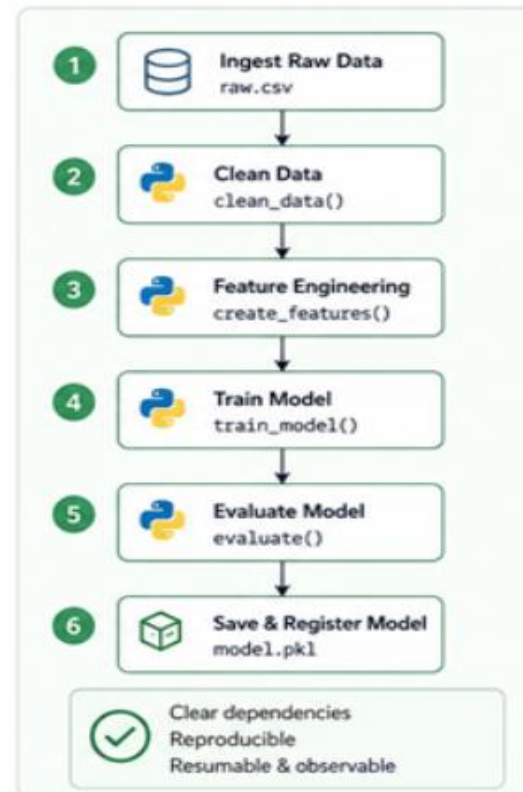


- **Enter Pipeline Orchestration:**
We need a framework to manage the flow of data, map the dependencies between our modules, and track execution state.

THE PROBLEM: SCATTERED & DISCONNECTED



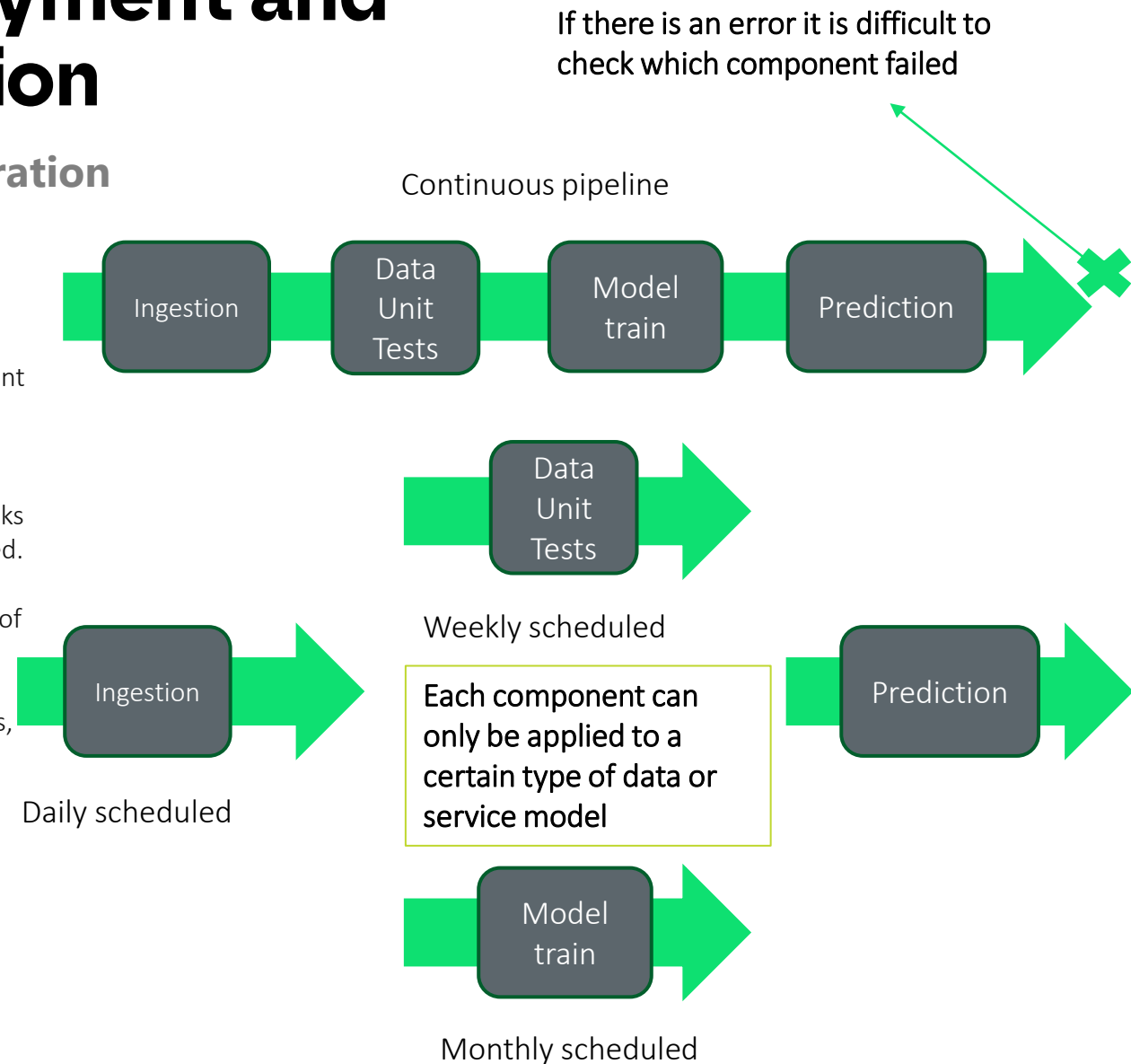
THE SOLUTION: ORCHESTRATED PIPELINE (DAG)



02: Model Deployment and Production

Pipelines orchestration

- **Efficiency & Automation:** Manual execution of ML tasks, from data preprocessing to model deployment, can be cumbersome and prone to errors. Orchestration automates these processes, reducing manual intervention and making the entire pipeline more efficient.
- **Reproducibility:** Orchestration tools, often combined with versioning tools, guarantee consistent reproduction of each step, from data processing to model training.
- **Scalability:** ML projects can grow in complexity, and the data involved can increase in volume. Orchestration tools help manage this growth by efficiently allocating resources and scaling tasks horizontally (across multiple machines) or vertically (using more powerful machines) as needed.
- **Consistency:** Orchestration ensures that every step in the pipeline adheres to a consistent set of practices and standards, leading to more predictable outcomes.
- **Error Handling & Recovery:** ML workflows can sometimes fail due to data issues, software bugs, or infrastructure problems. Orchestration tools detect failures, notify the appropriate parties, and even attempt to retry or recover from specific errors automatically.
- **Scheduling:** Some tasks in the ML pipeline might need to run at specific times or intervals. Orchestration tools allow for scheduling, ensuring that tasks like model retraining or data updates occur when they're supposed to.
- **Parallel Execution:** Some tasks within the ML workflow can run simultaneously because they don't depend on one another. Orchestration allows for parallel execution of these tasks, speeding up the workflow.



02: Model Deployment and Production

Pipelines orchestration using Kedro

- Kedro is a **development workflow framework** and a standard option for **developing production-ready code**. It was developed in QuantumBlack and McKinsey & Company and now hosted by the Linux Foundation (LF AI & Data).
- It was created **to ensure reproducible, maintainable, and modular code**, using the software engineering best practices on top of production ready data science code.
- Some successful applications of Kedro in real-world examples:
 1. **NASA** as part of their cloud-based predictive engine that predicts impeded and unimpeded taxi out duration within the airspace.
 2. **JungleScout** sped up the training and review of its sales estimation models 18 times with the help of Kedro.
 3. **Portuguese cyber security entity** uses it as part of the process for detecting phishing campaigns.
 4. **One of the main portuguese banks** on several use cases.
 5. **Tekmosel** uses Kedro to run hundreds of feature engineering tasks and serve dozens of machine learning models in their production environment.
 6. **ElementAI** improved their work efficiency by leveraging Kedro in their scheduling software to measure historical performance and create replay scenarios.

https://docs.kedro.org/en/stable/get_started/install.html Normal version

https://kedro-mlflow.readthedocs.io/en/stable/source/03_getting_started/index.html

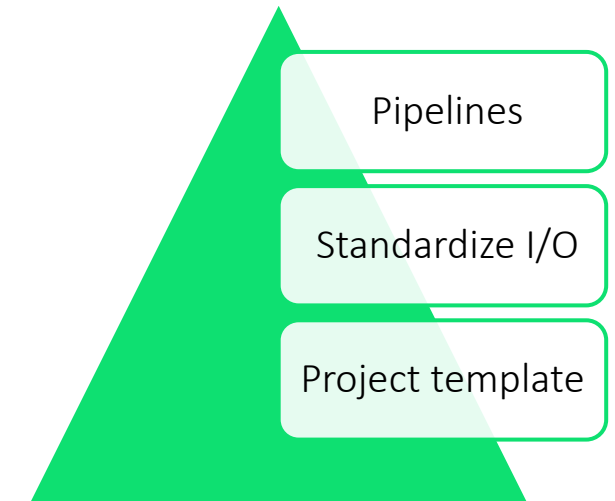
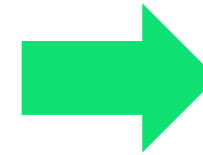
MLflow-plugin

02: Model Deployment and Production

Concepts behind Kedro

- Kedro tackles the following challenges:
 1. **Codes that need to be rewritten for production environments**, leading to significant project delays.
 2. **Project structures can be disorganized and incoherent**, making it challenging for collaboration.
 3. Data flow that is hard to trace.
 4. Functions that are overly lengthy and difficult to test or reuse.
 5. Relationships between functions that are hard to understand.

Kedro looks to Data Science project as a **acyclic graph** with three fundamentals:



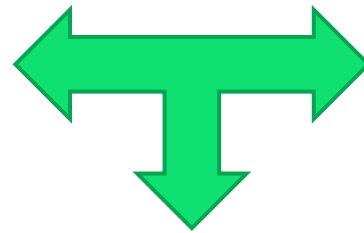
02: Model Deployment and Production

Concepts behind Kedro

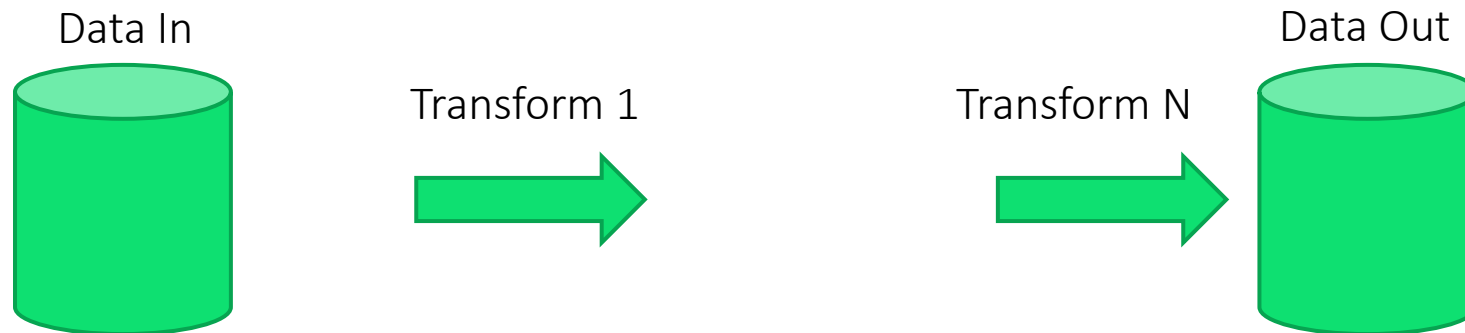
- A pipeline is a serial transformation of datasets which allow us to standardize each component of our Data Science pipeline.

Types of operations:

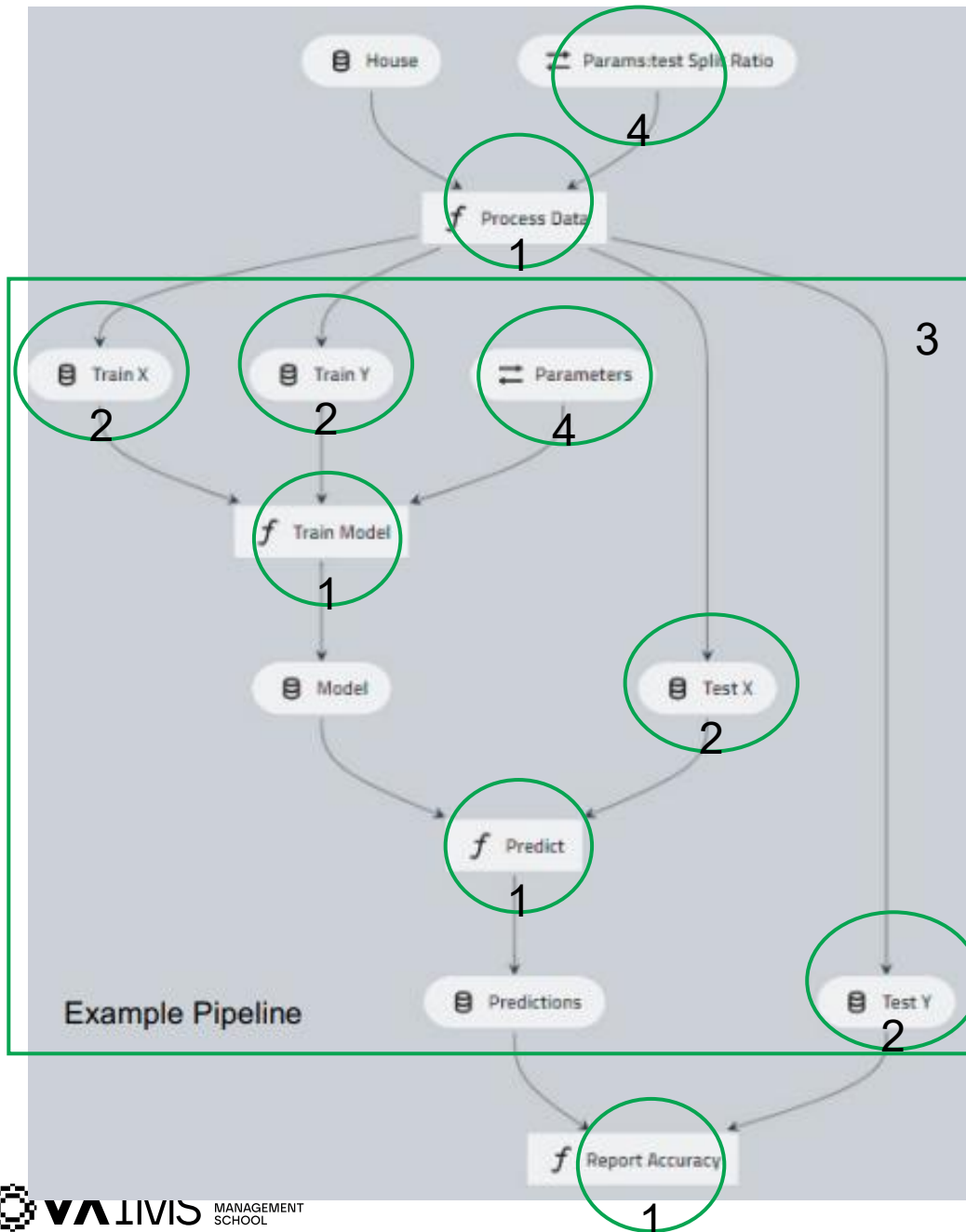
- **Reading/writing data**
- Transforming data, feature engineering ...



- **Independent modules** and easier to redirect a component .
- Code reusability and easier for debugging.



Kedro for real



1. **Node**, as a pure Python function that has inputs and outputs.
2. **Dataset**, as an impure Python function allowing reading/writing to storage. All datasets are registered in the **Catalog** file `catalog.yml`
3. **Pipeline**, a collection of nodes with defined relationships and dependencies. We can have a separated pipeline for data engineering, for model training and model evaluation.
4. **Parameter**, hard-coded variable where all parameters are specified in `parameters.yml`



Kedro provides a visualization tool to explore our pipeline structure and to export it as a .png file, (details when doing Demo class) by running "kedro viz" in the project's root directory.

02: Model Deployment and Production

Standard I/O

- A **yaml file as a Data Catalog**, which is an extendible collection of datasets and models .
The **purpose is to handle automatically several of operations**, using arguments from Pandas, Spark, ... :
 1. Read and write operations.
 2. Versioning.
 3. Remote connection, including authentication
 4. Compression.
 5. File header and schema.

```
catalog.yml x
9
10 bikes:
11   type: pandas.CSVDataSet
12   filepath: "data/01_raw/bikes.csv"
13
14 weather:
15   type: spark.SparkDataSet
16   filepath: s3a://your_bucket/data/01_raw/weather*
17   file_format: csv
18   credentials: dev_s3
19   load_args:
20     header: True
21     inferSchema: True
22
```

No read/write commands !!!

02: Model Deployment and Production

Concepts behind Kedro

- Standard project structure that eases collaboration and code maintenance:
 - conf:** Contains configuration files that specify vital details such as data sources (i.e. Data Catalog), model parameters, credentials, and logging information.
 - /data:** Contains the input, intermediate and output data. It is organized into an eight-layer data engineering convention to clearly categorize and structure how data is processed. (TBD on the following slide)
 - /docs:** Contains the files relating to the project documentation.
 - /logs:** Contains the log files generated when pipeline runs are performed.
 - /notebooks:** Contains Jupyter notebooks used in the project e.g., for experimentation or initial exploratory data analysis.
 - /src:** Contains the source codes for the project, such as Python scripts for the pipeline steps, data processing, and model training.

No more mix of production code and experimentation notebooks.
Hard-coded parameters in parameters.yml



02: Model Deployment and Production

Folders convention

Folder name	Definition
Raw	Contains the sourced data model(s) that should never be changed, it forms your single source of truth to work from. These data models are typically un-typed in most cases e.g. csv, but this will vary from case to case.
Intermediate	Optional data model(s), which are introduced to type your raw data model(s), e.g. converting string based values into their current typed representation.
Primary	Domain specific data model(s) containing cleansed, transformed and wrangled data from either raw or intermediate, which forms your layer that you input into your feature engineering.
Feature	Analytics specific data model(s) containing a set of features defined against the primary data, which are grouped by feature area of analysis and stored against a common dimension.
Model input	Analytics specific data model(s) containing all feature data against a common dimension and in the case of live projects against an analytics run date to ensure that you track the historical changes of the features over time.
Models	Stored, serialised pre-trained machine learning models.
Model output	Analytics of specific data model(s) containing the results generated by the model based on the model input data.
Reporting	Reporting data model(s) that are used to combine a set of primary, feature, model input and model output data used to drive the dashboard and the views constructed.

02: Model Deployment and Production

Other features

- Versatile CLI.
- Flexible Deployment: supports .egg or .whl packaging.
- **Kedro-Docker and Kedro-MLflow**: plugin to package Kedro projects in Docker containers and using MLflow.
- We can **generate project-specific documentation** (built on the Sphinx framework) by running "kedro build-docs" in the project's root directory.
- Kedro makes it easy to **set up experiment tracking, and access logged metrics from each pipeline run**. Besides its internal experiment tracking capabilities, Kedro integrates well with other MLOps services.
- **The pipeline slicing capabilities of Kedro allow us to execute specific portions of the pipeline** as we desire. For example, we can define the start and finish nodes in the pipeline slice that we wish to run (details when doing Demo class).

02: Model Deployment and Production

Alternative python packages



- Developed by Netflix.
- Metaflow is very well integrated with AWS.
- Metaflow pipeline works by steps.
- You can resume your workflow from where it failed.



- Developed by Spotify.
- Luigi pipeline has two building blocks, Task and Target.
- Tasks is a transformation, and the target is an output.
- Integration of task with other ecosystem jobs such as Spark.



- Developed by Apache/Airbnb
- Provides rich GUI with features including DAG visualization, execution progress monitoring, scheduling, and triggering.



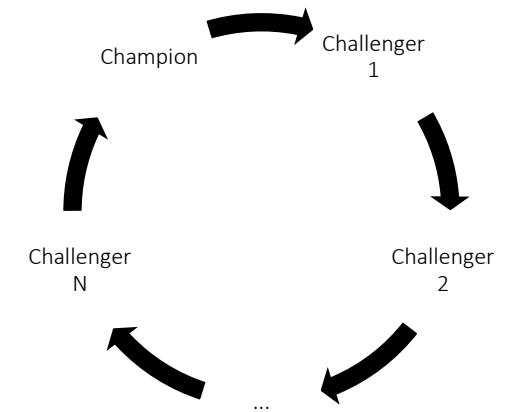
- Adds Python decorators to your existing code, ZenML automatically tracks experiments.
- You can easily switch between local and cloud orchestration.
- ZenML works with any known framework, whether it's classical machine learning or deep learning. Also, it is agnostic when compared to Cloud providers.

02: Model Deployment and Production

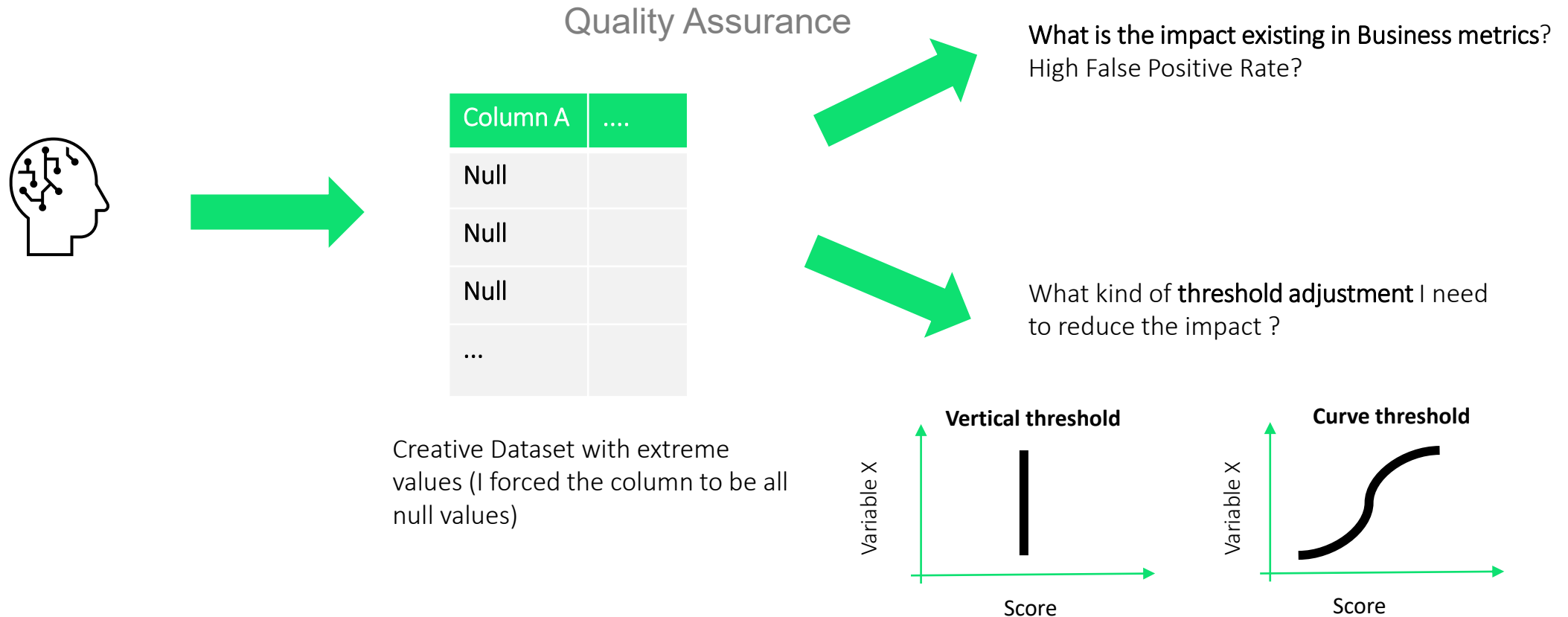
Quality Assurance

- Unlikely to software engineering, quality assurance methodologies in MLOps is still giving the first steps, and it is too much dependent on the degree of maturity of the data organization and appended teams.
- The entire process of data collection, data ingestion, data transformation, feature engineering model training and evaluation metrics should all be documented. It will be easier to validate every step of the process against organizational guidelines.
- Don't be spared about having assertions over your deploy pipeline. Create useful metrics as quality requirements for deployments.

Quality metrics	Model performance on production data (precision, accuracy, recall, e.g. an assertion on if the model accuracy falls below 90%). Metrics for computational performance (average latency, e.g. an assertion if the 5% of score events take more than x milliseconds).
Creative Scenarios	A feature (or more) that only has missing values. How does the performance changes? The distribution of several features changes (one can force some events to have extreme values). Can the process be used for 3x times the quantity of the expected data.
Subpopulation analysis and model fairness	Overall metrics of the model are very good. But splitting the data into subpopulations it seems that there exists a concentration of false positives in a specific segment.



02: Model Deployment and Production



Even I have a strong layer of data unit tests, there will be some time to react. Meanwhile I need to have some mitigation measures.

02: Model Deployment and Production

Real-world examples



Ecommerce card fraud detection:

- Clients without recent history of usage have a much more higher rate of rejected first transaction (this can affect user experience).
- Trained dataset without any period of holidays peak season -> high false positive rate on transactions for travel agencies, flight companies...
- Trained dataset sampled with the majority of fraud highly correlated with the age and education of victims – blocking certain groups can affect bank reputation.



Healthcare:

- Prediction of patients that would likely need extra medical care.
- There was a feature highly correlated to race was, which was healthcare cost history.
- The rationale was that cost summarizes how many healthcare needs a particular person has.
- For various reasons, black patients incurred lower healthcare costs than white patients with the same conditions on average.



Vehicle stock and sales recommendation:

- Model developed for a specific market.
- When the model was world-wide scaled, could not handle efficiently the new amount of data.

02: Model Deployment and Production

Risk

- Production data changes for some reason (world war, seasonality, bug on a system). The value of a feature at inference time is out of the bounds. Should we have process where the system can trigger appropriate measures, such as rejecting the sample or dispatching a warning message?
- In a classification problem, each event is scored and a threshold is defined for the final decision. Facing any seasonality event, one can have some dynamic threshold that will be responsible for accepting an inference output, or anomaly detection algorithm to find the events that are outside of the application domain.
- Model with a high cardinality of feature will be more sensitive to these issues: the number of combinations is exponential, so the performance could be unpredictable.
- Is the application domain of a new model already occupied by another model or heuristic rules? Should the outcome be combined? What strategy should be done to switch models, if it's the case?

Security

- There should be several access layers to the training dataset and model output. Models can leak data, for example, a model predicts how much someone is paid using the nearest neighbor algorithm. If one knows the zip code, age, and profession of a certain person registered on the service, the leak will happen...
- Connections to the external environment of the organization. If we need enrich the data while the model acting on real time, there is a chance an attacker be able to change the training data or to introduce small perturbations on the production data.

